

Globant preparation Junior

Seniority	Technical	Soft	Studio
Jr	Math and Stats knowledge, Machine Learning fundamentals	Owning the assigned task and driving requests for help.	
SSr	Solid knowledge of a basic set of algorithms in some platform, focusing on data analysis.	Given the assigned task, executing correctly with independence, already some experience in the DS role.	Participates as support in presales, aids in content generation and studio tasks.
Sr	Solid knowledge of Classic ML + DL, GenAI. Experience with Agile software development.	Mapping business problems to AI solutions, defining the right task, explaining rationale to different profiles. Proactive in seeking feedback, effective client facing, solid experience as DS.	Interviews, leads presales, content generation.

High Seniority	Add to the above knowledge of architecture of modules and data flow, solving strategies, modularization, re-risking.	Coordinate project vision and definition, extensive experience in the DS role.	Guardian of relevance, acts as ambassador and PR agent to the world, interviews.
----------------	--	--	--

Agility

"Agile" is a term to describe a set of methodologies widely adopted nowadays for software development. One of its main tenets is to move forward in small time iterations or "sprints", which result in deliverable software which incrementally increases in capability. These practices are many times a hard match for a discipline like data science, which hinges on research and explorations whose outcomes are often unforeseeable at the beginning of an iteration. That much is true. It will not, however, be used as an excuse for not being "more agile" in the way we work:

- Plans are useless but planning is indispensable. The aim is to get the structure of what we want to build and the road to get there, not the exact result and date.
 - In data science, knowledge is also a deliverable and is just as valuable as a piece of software. But anything you have found has to be properly documented! Stating your findings verbally or showing an untidy notebook does not count.
 - What can you do "for tomorrow EOD" that still delivers something of value? How does that help the business? Iterative and incremental!
 - "Quick wins" sound like burning paper instead of coal. Try to turn them into "early returns" upon which to build the next phase.

Think of Agile as gradient descent. It will often let you make more appropriate decisions instead of continuing to search for the perfect option on a sparse quantity of more "wicked" problems. In this view, agility serves us as a "discipline enforcer" that keeps us from going too deep when is not warranted, does not let us just browse around, and who makes sure we keep focused on the expected value to be delivered rather than going down a rabbit's hole of "cool" algorithms.

For us, then, the key Agile skill is how to break down our work into manageable chunks that are still meaningful. Every User Story should have a deliverable of sorts: this could be a working version of code (preferably), a document with the analysis and actionable conclusions that derive from it, a report on the plan to follow, or anything else that creativity allows us. But whatever it is, it needs to deliver value; either as software or gained knowledge.

In the same vein that a minimum viable product (MVP) represents the minimum

set of features so that a software delivers value to users, we have to think about the minimum data science work required to gain a new insight or functionality. Any large research and development can be broken down into a myriad of these smaller units, often parallelizable to leverage a larger

Junior

As a Junior Data Scientist you should know about and be able to explain in relatively simple terms the following topics:

Statistics

- Big samples vs small samples? Pros and cons.
 - Explain metrics of central tendency and dispersion:
 - Mean, mode, median (When do you prefer a median to the mean? When is a mean not representative? Etc).
 - Variance / standard deviation, L1 distances, Kurtosis (When is standard deviation not a good indicator of at range? Etc).
 - Data distributions and data visualizations:
 - Types of plots: histogram, box plot, scatter plot, bar chart, etc (When and why would you use each type of plot?)
 - Cumulative distribution functions (What do these functions tell you?)
 - Multimodal distributions (What does that mean and imply?)
 - Name some distribution functions and logic from which they arise (e.g. Gaussian, LogNormal, Weibull, Gumbel, Uniform, Triangular, Gamma, Beta, Poisson, Binomial...)
 - Validation and hypothesis testing:
 - P-value (What does it mean, and how reliable is it?)
 - Hypothesis testing (What is it, and why would you need it?)
 - Name some different sampling procedures (e.g. random, stratified, Poisson disc, among others).

Math

- Matrix multiplication, identity, inversion, determinant, chain of products, systems of equations.
- Derivatives, integrals, limits

Code

- Python; the fundamentals, focusing on data science / data visualization / machine learning libraries.
 - Data types and differences / use cases (list, dict, set, tuple, numpy.ndarray, pandas.dataframe, etc)
 - What is aliasing? Why is it dangerous when unchecked?
 - What is the use for `_name_=="_main_"`?
 - Virtual environments: venv, uv, conda, poetry. What are they and why use them?
 - Fundamental data science libraries: pandas, numpy, scikit-learn.
 - Data visualization: Matplotlib, Seaborn. Plotly. Dash, etc

Semi Senior

As a Semi Senior Data Scientist you should be able to expand on the above by knowing about the following topics:

Classical Algorithms

- Linear regression
 - Logistic Regression
- Decision Tree
- Random Forest
- k-Means
- KNN
- Shallow NN
- SVM
-

Statistics / Math

- Central Limit Theorem (Can you explain it in simple terms? Why is it useful?)

Cross validation (Explain the technique., Why do we need it? what is data leakage? why partition data?)

Metrics for classification problems: confusion matrix, accuracy, precision, recall, F1 score.

Metrics for regression problems: MAE, MAPE, RMSE.

- Gradient descent (How does it work? Soft requirement on convexity? When does it not matter?)
- What is a convolution?
- Hypothesis generation.
- Error analysis.
- Explain how the autocorrelation of a positive-trend line can be negative.
- ANOVA, multivariate analysis, PCA / Factor Analysis, population tests and comparison.

Machine Learning

- Algorithm customization
 - What do you do if you are trying to do regression and you have heteroskedasticity?
 - How do you avoid overfitting?
 - How do you add non-linearities to a regression?
 - How do you add classes to a regression?

Variable selection / feature engineering (What is it? How would you do it?)

- Handling "too many features" (Is that a problem? When? What would you do about it?)

How can you choose the value of k on k-means?

Transformer Basics: Understands the "Encoder-only" (BERT) vs. "Decoder-only" (GPT)

Understands Embeddings as an extension to "Feature Engineering."

LLM

Workflow vs. Agent: Understands the difference between a fixed Workflow (Step A -> Step

B) and a dynamic Agent (Goal -> Reason -> Action).

-Vector DB & other types of Retrieval: identifying when to use each based on trade-offs in latency, scale, and metadata filtering requirements.

Encoding & Token Awareness: Understands how different models "count" tokens and how that impacts both cost and performance.

- Context Precision & Recall: Did we retrieve only relevant documents? Did the retrieval step actually include the document containing the answer?

Add context to the model using prompting techniques: Zero and few shots, prompt chaining, etc.

Software Development

- What is the difference between "checkout", "add", "commit" and "branch" in Git?

- What are the 4 main parts of an SQL statement?

Python

- Basic API development: FastAPI, Flask, Eventlet / Gunicorn. REST, wsgi.

- Visualizations: Bokeh, Plotly, others.

Fundamentals of Object Oriented Programming.

Packaging classes and methods into data science libraries.

Literate programming.

Reproducible research.

Error catching, context manager, generators and comprehensions, reusable methods and classes, some code optimization and profiling.

Basic Python Engineering: Able to write clean functions that can be used as "Tools" by an AI.

- Basic Agent Tool: LangChain, Smolagents, etc.

<https://scholar.google.com/citations?user=6dskOSUAAAAJ&hl=en> : https://ai-plans.com/file_storage/4f32fa39-3a01-46c7-878e-c92b7aa7165f_2212.08073v1.pdf

LLM and Generative AI

-Knows and use of generative AI-based tools.

Basic prompting techniques. Prompt Hygiene.

Senior

As a Senior Data Scientist, you should expand your portfolio of skills to add the following:

General AI Knowledge

Fields of AI

- Machine Learning
 - Simulation
 - Discrete Event Simulation

I

- Agent Based Modelling
 - System Dynamics / Numerical Partial Derivative Systems
 - Optimization
 - Mixed Integer Programming and variations

Computer Vision / Signal Processing

Natural Language Processing

Generative AI

Deep Learning

Layers in a neural network (Fully Connected, Convolutional, Recurrent, Dropout, Pooling, Batch

Normalization, Attention)

Learning schemas (SGD, Adam, adaptive models, batch size/ learning rate tradeoff etc)

Loss functions (L-based distance, cross-entropy, others)

- Evaluation
 - Bias / Variance tradeoff

Epoch graph vs Dataset Size graphs

- Error analysis
 - What is it? What options do you have?
 - What are the options when an NN does not give good results? Why?

Variable normalization / feature scaling

Is linear separability a requirement before using logistic regression? Or is it desirable?

SVMs: pros and cons

NN

ReLU

NLP

-Explain TDF-IDE, LDA, Bag of Words.

LLMs / GenAI / Agents

MoE (Mixture of Experts): Understanding how models like GPT-4 or Mixtral activate only specific "expert" sub-networks to save on inference costs.

Fine-Tuning (PEFT/LoRA): Knowing the math of "Low-Rank Adaptation"-how to update a tiny fraction of weights to specialize a model without retraining the whole thing.

-Knowledge Graphs: Moving beyond Vector DBs to graph-based relationships for complex

RAG.

KV Caching: Understanding how the model "remembers" previous tokens in a session to save compute power and reduce latency.

Guardrails & Safety: Implementing NeMo or LlamaGuard (Understanding the "Alignment" problem).

LLM-as-a-Judge (G-Eval): Building a deterministic rubric for an LLM to grade another LLM.

Agentic Frameworks & Patterns: Mastery of orchestration patterns like Evaluator-Optimizer, Routing, and Parallelization.

- Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to improve retrieval using Knowledge Graphs.
- Transcript Evaluation: Analyzing the path the agent took (did it waste tokens? did it get stuck in a loop?).
- Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g., "Was the flight actually booked?").
- Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?
- Agentic AI vs. AI Agents: Can distinguish between building a single autonomous script (Agent) and building a system where AI is embedded into the whole business process

Epoch graph vs Dataset Size graphs

Error analysis

-What is it? What options do you have?

- What are the options when an NN does not give good results? Why?

How do you avoid the infamous racial bias on object detection (detecting a family of African-Americans as apes)

What do algorithms like Word2Vec (or ULMFit, or encoder LLMs) do? How does it work?

- Define "embeddings"
 - Why "negative sampling"?
- Pros cons when modeling sequence

- LSTM
 - 1d convolution

Explain differences, and how the approaches differ, e.g. with "objects"

Detection

Semantic segments

Instance segments

Machine Learning

- Class imbalance (What is the issue? How would you solve it?)
 - Explain pros and cons of the different possible models based on the final objective of the model and possible side-effects, without resorting to "try it out and see" on the data Eg. Without having the data (so "trying it out" is not an option), why would you choose, or what would you take into account, to choose among the following models

-Logistic

Tree

Forest

NN

K-NN

- Variable normalization / feature scaling
 - Is linear separability a requirement before using logistic regression?
Or is it desirable?
 - SVMs: pros and cons

NLP

- Explain TDF-IDF, LDA, Bag of Words.

LLMs / GenAI / Agents

- MoE (Mixture of Experts): Understanding how models like GPT-4 or Mixtral activate only specific "expert" sub-networks to save on inference costs.

Fine-Tuning (PEFT/LoRA): Knowing the math of "Low-Rank Adaptation"-how to update a tiny fraction of weights to specialize a model without retraining the whole thing.

-Knowledge Graphs: Moving beyond Vector DBs to graph-based relationships for complex

RAG.

KV Caching: Understanding how the model "remembers" previous tokens in a session to save compute power and reduce latency.

Guardrails & Safety: Implementing NeMo or LlamaGuard (Understanding the "Alignment" problem).

LLM-as-a-Judge (G-Eval): Building a deterministic rubric for an LLM to grade another LLM.

Agentic Frameworks & Patterns: Mastery of orchestration patterns like Evaluator-Optimizer, Routing, and Parallelization.

- Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to improve retrieval using Knowledge Graphs.
Transcript Evaluation: Analyzing the path the agent took (did it waste tokens? did it get stuck in a loop?).
Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g., "Was the flight actually booked?").
 - Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?

Agentic AI vs. AI Agents: Can distinguish between building a single autonomous script (Agent) and building a system where AI is embedded into the whole business process

Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to

improve retrieval using Knowledge Graphs.

Transcript Evaluation: Analyzing the path the agent took (did it waste tokens?)

did it get stuck in a loop?).

Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g),

"Was the flight actually booked?").

Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?

Agentic AI vs. AI Agents: Can distinguish between building a single autonomous script (Agent) and building a system where AI is embedded into the whole business process

(Agentic AI).

-Agent frameworks: LangChain/LangGraph, AutoGen, CrewAI, among others.

Optimization

- Solving speed on a MIP (Mixed-Integer Programming) solver

If you want to speed up solving time, what do you change? Objective function,

- variables, constraints?
- What is the trade-off of heuristics on optimization models?

Development

- How would you handle an Agile methodology for doing data science?
 - Why not just ask people instead of modelling preferences to predict behavior?
 - Be able to state explicit hypotheses to validate through data (How does the world work?)
 - What don't I know about the world? -> Discover through data
 - What are the uncharted territories? -> Use data as an exploration
 - The art of breaking up a project into User Stories
 - How to generate a wealth of hypothesis and interpretations from a dataset /
 - visualizations / a problem statement / a business context

Higher Seniorities

Beyond Senior Level 1, you will be expected to:

- Go above and beyond the strict data science tasks: manage larger projects and teams, deal competently with the broader data engineering space..
 - Be ready to engage with a broad audience: peers, clients, conferences, etc.
Communication and presentation skills become critical at this level.
 - Care about how the client will grow, not just the project or account.
That makes us different.
 - Not limit yourself to your data science skill set, but look through the clients' eyes at their problems and understand their vision.
Always keep in mind that while we may not be sales people, we team up with them to reach our clients. We have a data-driven insight no

one else will have. We should become our client's consiglieri (Guibert dixit). It's not about selling, it's conveying what we can and what we like to do. It's saying "yes and" or "no but".

- Communicate our vision about AI.
- Be mindful of AI business strategy, processes, governance, and incorporate such topics in projects and proposals.

•

•

Be up to date in our Technology Strategy regarding AI, platforms, evolvability. Take up a mentoring role, but:

- You're meant to be a passive, not active, guide for other Globers.

You are an advisor, not responsible for the mentee's actual progress.

Mentoring is part of the usual studio-related tasks just like presales or generating material, and should be conducted likewise.

Treat people as future peers. Respect, empathy and consideration are always relevant.

Bayesian vs frequentist

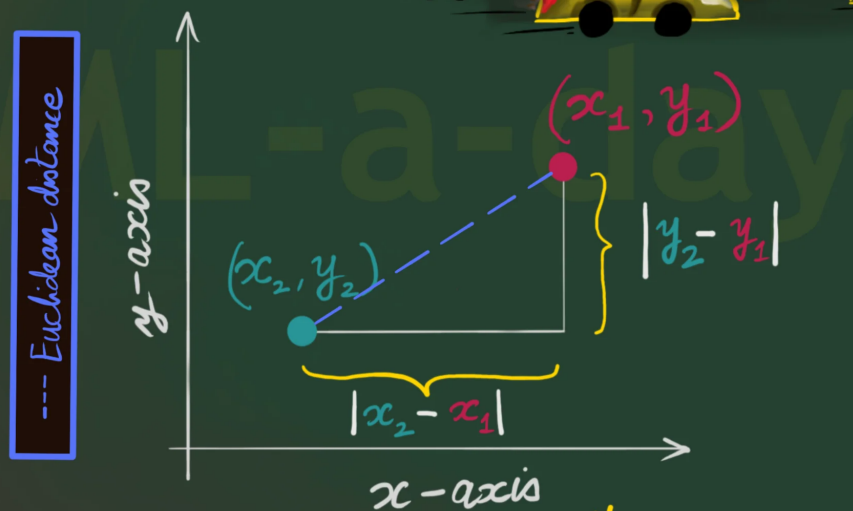
LSTM

Mathematical optimisation

adversarial examples and grokking,

Repaso Junior:

L_1 distance also known as the Manhattan / Taxicab distance as it is the shortest distance in the right-angled streets of NYC.



$$L_1(p, q) = \|p - q\|_1 = \sum_{i=1}^N |p_i - q_i|$$

There are many possible norms on $\mathbb{R}^n$.

Examples: consider a vector $\mathbf{x}$ in $\mathbb{R}^n$ with elements $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Then:

- ① The Euclidean distance, or l_2 norm:

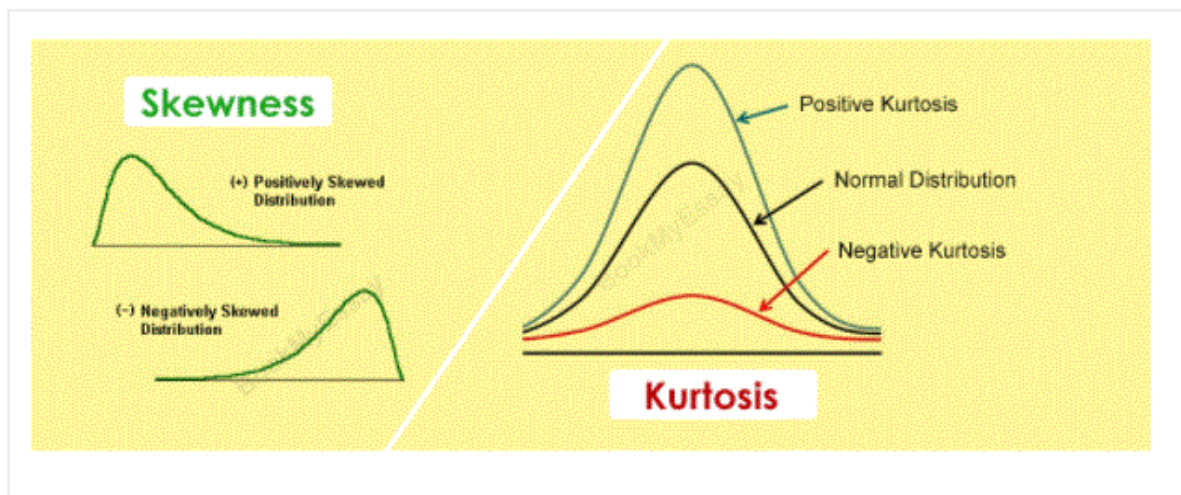
$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

- ② The l_∞ or *sup* norm:

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

- ③ The l_1 norm:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$



Standard deviation (SD) is not a good indicator of range or spread when the data is **skewed**, contains **outliers**, or follows a **non-normal distribution**. Because SD is calculated by squaring the distance of each point from the mean, extreme values are disproportionately weighted, which can lead to a misleading representation of the "typical" spread.

1. Histogram

- **Purpose:** Shows the frequency distribution of a continuous variable by grouping data into intervals (bins).
- **Best for:** Visualizing data distribution, skewness, spread, and normality.
- **Example:** A histogram showing the frequency of exam scores within specific ranges (0-10, 10-20, etc.).

2. Box Plot (Box-and-Whisker Plot)

- **Purpose:** Summarizes data distribution using five-number summaries: minimum, first quartile (Q1), median, third quartile (Q3), and maximum.
- **Best for:** Comparing distributions between groups, identifying outliers, and visualizing variability.
- **Example:** Comparing the distribution of salaries (y-axis) across different departments (x-axis).

3. Scatter Plot

- **Purpose:** Displays individual data points for two continuous numerical variables to identify relationships or correlations.
- **Best for:** Determining the strength, direction (positive/negative), and shape of relationships, as well as detecting clusters or outliers.
- **Example:** Plotting advertising spend (x) against sales revenue (y) to see if higher spend correlates with higher revenue.

4. Bar Chart (Bar Graph)

- **Purpose:** Uses rectangular bars to compare numerical values across

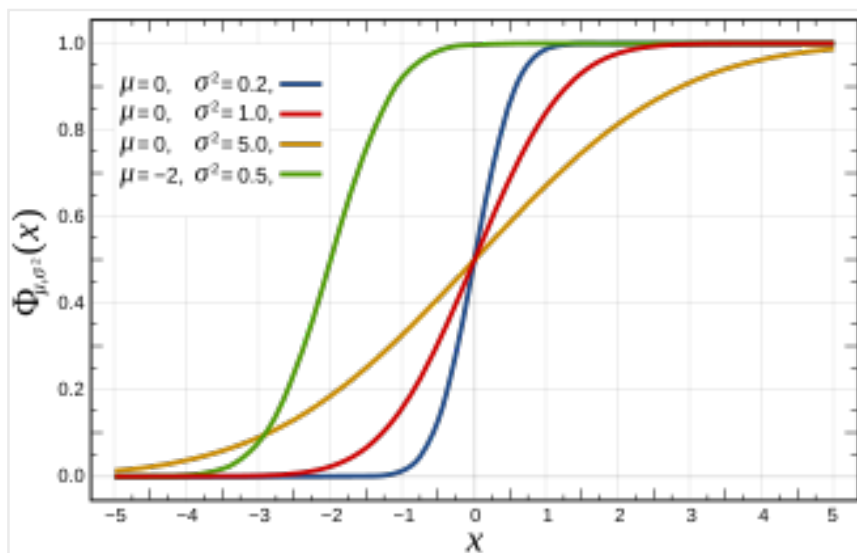
different categorical groups.

- **Best for:** Comparing nominal data, counts, or nominal totals.
- **Example:** A bar chart comparing total sales (numerical) by region (categorical: North, South, East, West).

Key Differences:

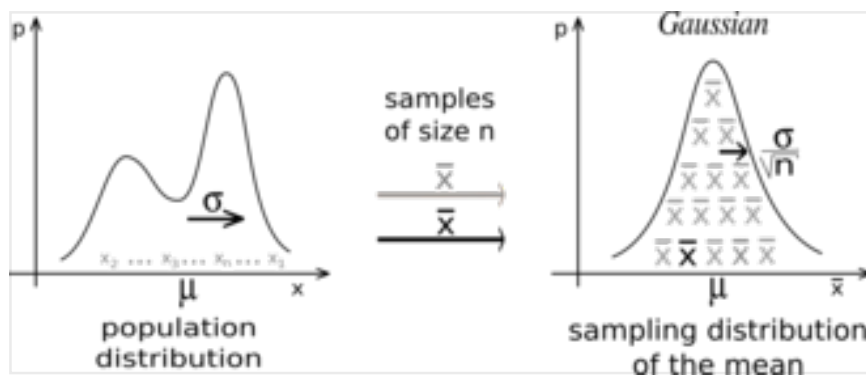
- **Histogram vs. Bar Chart:** Histograms group continuous data into ranges (bins), while bar charts represent discrete, categorical, or nominal data.
- **Histogram vs. Box Plot:** Histograms show the full shape of the distribution, while box plots are more compact and better for comparing multiple groups or identifying outliers.

Cumulative distribution function for the **normal distribution**

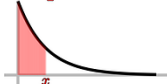
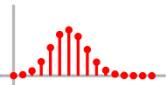
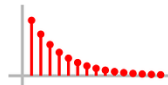
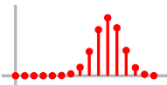


Here is a breakdown of common probability distributions and the logic from which they arise:

Probability distributions arise from specific mathematical "stories" or logical constraints that describe how data is generated. Understanding these underlying logics helps in choosing the right model for real-world phenomena.



Multimodal: when we have two or more clusters

Probability Distributions				 Ace Tutors
Continuous	Uniform  $\mu = \frac{a+b}{2} \quad \sigma = \sqrt{\frac{(b-a)^2}{12}}$ $P(X < x) = \frac{x-a}{b-a}$	Exponential  $\mu = \frac{1}{\gamma} \quad \sigma = \frac{1}{\gamma}$ $P(X < x) = 1 - e^{-\gamma x}$	Normal  $z = \frac{x - \mu}{\sigma}$ $P(X < x) \Rightarrow \text{Use Z-Chart}$	Key γ = rate parameter z = z-score p = probability of success n = # of trials N = population size K = # of success states
	Binomial  $\mu = n \cdot p \quad \sigma = \sqrt{n \cdot p \cdot (1-p)}$ $P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}$	Geometric  $\mu = \frac{1}{p} \quad \sigma = \frac{\sqrt{1-p}}{p}$ $P(X = x) = (1-p)^{x-1} p$	Hypergeometric  $\mu = n \frac{K}{N} \quad \sigma = \sqrt{n \frac{K(N-K)(N-n)}{N^2(N-1)}}$ $P(X = x) = \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$	

Probability distribution functions are mathematical models that describe the likelihood of different outcomes for a random variable. They arise from specific logical frameworks based on how data is generated, aggregated, or limited by physical or statistical constraints.



Wiley Online Library

+4

Here is a breakdown of common distributions and the underlying logic from which they arise:

1. Gaussian (Normal) Distribution

- **Logic: Additive processes (Central Limit Theorem).** When a random variable is the sum of a large number of small, independent, identically distributed (i.i.d.) random variables, it tends toward a Gaussian

distribution, regardless of the original underlying distribution.

- **Phenomena:** Physical characteristics (height, weight), measurement errors, test scores.
- **Key Idea:** Clustering around a central mean with symmetric, "thin" tails.

• Brilliant
+4

2. LogNormal Distribution

- **Logic: Multiplicative processes.** If a variable is the result of multiplying many small, independent, positive random variables, its logarithm follows a normal distribution (just as sums lead to normal, products lead to lognormal).
- **Phenomena:** Incomes, sizes of particles (breaking/fracturing), reaction times, lengths of inert appendages.
- **Key Idea:** Skewed right, with a long tail on the positive side.

In [probability theory](#), a **log-normal** (or **lognormal**) **distribution** is a continuous [probability distribution](#) of a [random variable](#) whose [logarithm](#) is [normally distributed](#).

3. Weibull Distribution

- **Logic: Weakest-link theory (Extreme Value Theory).** It represents the distribution of the *minimum* of a set of random variables. It is used to model lifetime data where the system fails when its weakest component fails.
- **Phenomena:** Material strength, fatigue life, reliability modeling, wind speed.
- **Key Idea:** Highly versatile; can model decreasing, constant, or increasing failure rates based on shape parameters.

• ScienceDirect.com
+4

4. Gumbel (Generalized Extreme Value) Distribution

- **Logic: Asymptotic distribution of maximums.** It arises when considering the maximum value of a large number of samples from a distribution with a "thin" tail (like the exponential or normal).
- **Phenomena:** Modeling maximum daily rainfall, river flood levels, or stock market extremes.

5. Uniform Distribution

- **Logic: Equally likely outcomes.** Every possible value within a range

-

has the same probability density.

- **Phenomena:** Random number generation, dice rolling, rounding errors.
- **Key Idea:** Constant probability across the entire range.

6. Triangular Distribution

- **Logic: Bounded uncertainty with subjective knowledge.** Used when only the minimum, maximum, and most likely (mode) values are known, often in project management or risk analysis.
- **Phenomena:** Estimated project completion times, cost projections.
- **Key Idea:** Linear increase to a peak, followed by a linear decrease.

7. Gamma Distribution

- **Logic: Sum of exponential times.** Models the time required for a total of independent events to occur, where each event happens at a constant average rate.
- **Phenomena:** Waiting times, rainfall totals, lifetimes of components with wear-out periods.
- **Key Idea:** Flexible shape (often skewed) for modeling waiting times or positive skewness.

8. Beta Distribution

- **Logic: Probabilities of probabilities.** Models a random variable that is constrained to a fixed interval, such as , making it ideal for modeling uncertain probabilities or percentages.
- **Phenomena:** Proportions (e.g., market share, conversion rates), Bayesian updating of binomial outcomes.
- **Key Idea:** Extremely versatile shape, from U-shaped to bell-shaped.

9. Poisson Distribution

- **Logic: Discrete events in fixed time/space.** Models the number of independent, rare, or sporadic events occurring within a specific interval, given a known average frequency.
- **Phenomena:** Number of customer arrivals per hour, radioactive decays per minute, accidents per year.
- **Key Idea:** Discrete (counts); mean equals variance.

10. Binomial Distribution

- **Logic: Sum of Bernoulli trials.** Represents the number of successes in a fixed number of independent, binary (yes/no) trials, where each trial has the same probability of success
- **Phenomena:** Coin flips, defective items in a batch, pass/fail results.
- **Key Idea:** Discrete; models the count of "successes"

Hypothesis testing (What is it, and why would you need it?)

Statistical hypothesis testing involves making a decision about two competing hypotheses. The null hypothesis (H_0)

is a statement about the assumed value of a population parameter. It is usually a hypothesis about no difference or no relationship. The alternative hypothesis (H_1)

is a statement about the value of a population parameter that you want to test. It is usually a hypothesis that has some difference or some relationship.

1. Determine the null (H_0) and alternative (H_1) hypotheses. The null hypothesis is assumed to be true when you start your analysis. It is the logical opposite of your suspicion.
2. Select a significance level. The significance level is the amount of evidence needed to overturn your assumption that the null hypothesis is true.
3. Collect evidence (data).
4. Use a decision rule to make a judgment. If the evidence in the data is sufficiently strong, based on the selected significance level, then reject the null hypothesis. If the evidence in the data is not strong enough, fail to reject the null hypothesis. It is important to note, however, that failing to reject the null hypothesis does not prove the alternative hypothesis.

What is a p -value?

A reference distribution enables you to quantify the probability of observing a particular outcome (the calculated test statistic) or a more extreme outcome if the null hypothesis is true. That probability is called the p -value.

A large p -value indicates a high probability of observing your results or more extreme results, given that H_0

is true. Therefore, it is reasonable to continue to assume H_0

is true, and you fail to reject the null hypothesis. A small p -value indicates a low probability of observing your results or more extreme results, given that H_0

is true. Therefore, it is no longer reasonable to assume that H_0

is true, and you reject the null hypothesis.

The p -value is a number between zero and one, inclusive. It is a probability that is calculated from your data.

The null hypothesis is often described as "negative" because it typically represents a position of **no effect**, **no difference**, or **no relationship** between variables. It acts as the default "status quo" or "nothing" hypothesis that researchers aim to test, or "disprove," through statistical evidence

Name some different sampling procedures (e.g. random, stratified, Poisson disc, among others).

Sampling Procedure	Description	Advantages	Disadvantages	Typical Applications
Random Sampling	Each sample has an equal probability of being selected.	Simple to implement; statistically unbiased; easy to analyze; good baseline method.	Can produce clusters or gaps; poor spatial coverage; higher variance in heterogeneous environments.	General statistics, ML dataset splitting, baseline ecological studies.
Stratified Sampling	Population divided into strata (groups) and sampled within each stratum.	Ensures representation of different groups; reduces variance; good for heterogeneous populations.	Requires prior knowledge to define strata; incorrect stratification may introduce bias; more complex design.	Ecology, habitat studies, socioeconomic surveys, environmental monitoring.

Systematic Sampling	Samples collected at regular intervals (e.g., grid or every k units).	Good spatial coverage; easy to implement; efficient in field surveys.	Can introduce bias if periodic patterns exist in the data; randomness is limited.	Forestry surveys, agricultural monitoring, spatial sampling.
Cluster Sampling	Population divided into clusters and some clusters are sampled entirely.	Cost-efficient when populations are geographically dispersed; reduces travel/logistics costs.	Higher sampling error; clusters may not represent the whole population well.	Large-scale field surveys, household surveys, ecological monitoring.
Poisson Disc Sampling	Random sampling with a minimum distance constraint between samples.	Produces evenly distributed samples; avoids clustering; good spatial coverage; useful for spatial analysis.	More complex algorithm; computationally heavier; not purely random.	Computer graphics, spatial ecology, environmental monitoring.
Latin Hypercube Sampling	Multidimensional sampling ensuring coverage across each variable range.	Efficient exploration of parameter space; fewer samples needed than random sampling; good for simulations.	More complex to design; not ideal for spatial constraints alone.	Sensitivity analysis, simulation studies, environmental modeling.

Adaptive Sampling	Sampling intensity changes depending on observed values (e.g., more samples where phenomena occur).	Efficient for detecting rare or clustered events; focuses effort where needed.	Harder statistical inference; requires dynamic design during sampling.	Rare species surveys, ecological hotspot detection.
--------------------------	---	--	--	---

Math

- Matrix multiplication, identity, inversion, determinant, chain of products, systems of equations.

3. Matrix Multiplication

Matrix multiplication combines transformations.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

Multiplication rule:

Row of A × Column of B

Example first element:

$$(1)(2) + (2)(1) = 4$$

Result:

$$AB = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

Important properties:

- **Not commutative**

$$AB \neq BA$$

5. Matrix Inverse

The **inverse matrix** reverses a transformation.

Definition:

$$AA^{-1} = I$$

Example:

If matrix **A rotates and scales**, the inverse **undoes that transformation**.

Example:

$$A^{-1}$$

takes transformed vectors back to their original position.

Not all matrices have an inverse.

6. Determinant

The **determinant** is a number describing how a matrix **scales space**.

For a 2×2 matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Determinant:

$$\det(A) = ad - bc$$

Geometric interpretation:

The determinant represents the **area scaling factor**.

Example:

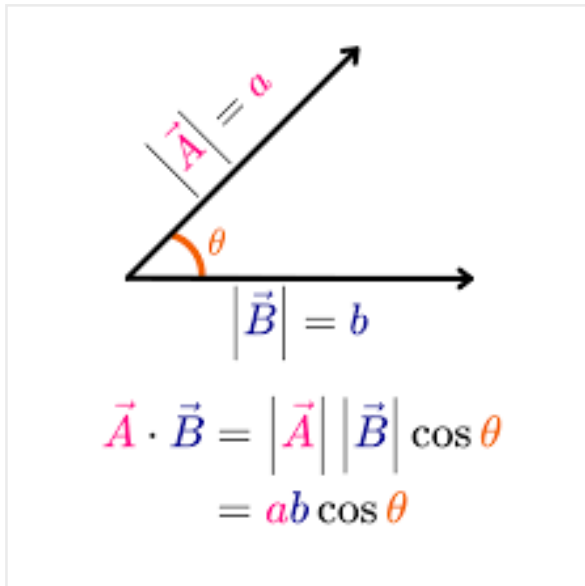
- $\det(A) = 2 \rightarrow$ area doubles
- $\det(A) = 0 \rightarrow$ space collapses to a line

If:

$$\det(A) = 0$$

the matrix **cannot be inverted**.

Producto punto



$$\vec{A} \cdot \vec{B} = AB \cos(\theta)$$

$$= A_x B_x + A_y B_y + A_z B_z$$

El producto punto es una manera fundamental en la que podemos combinar dos vectores. De manera intuitiva, nos dice algo acerca de qué tanto apuntan dos vectores en la misma dirección.

Matrix chain multiplication (or the **matrix chain ordering problem**^[1]) is an **optimization problem** concerning the most efficient way to **multiply** a given sequence of **matrices**. The problem is not actually to *perform* the multiplications, but merely to decide the sequence of the matrix multiplications involved. The problem may be solved using **dynamic programming**.

There are many options because matrix multiplication is **associative**. In other words, no matter how the product is **parenthesized**, the result obtained will remain the same. For example, for four matrices A , B , C , and D , there are five possible options:

$$((AB)C)D = (A(BC))D = (AB)(CD) = A((BC)D) = A(B(CD)).$$

-Derivatives, integrals, limits

1. Limits (Límites)

Meaning

A **limit** describes what value a function **approaches** when the input gets closer to some number.

Example:

$$\lim_{x \rightarrow 2} f(x)$$

2. Derivatives (Derivadas)

Meaning

A **derivative** measures **how fast something changes**.

It represents the **rate of change** of a function.

Geometric meaning

The derivative is the **slope of the tangent line** to the curve at a point.

So it tells us:

- how steep the curve is
- whether it is increasing or decreasing

Example interpretations:

Situation	Meaning of derivative
Position vs time	Velocity
Velocity vs time	Acceleration
Population vs time	Growth rate
Temperature vs time	Rate of change of temperature

3. Integrals (Integrales)

Meaning

An **integral** measures **accumulation** or **total quantity**.

Example:

If a function represents **speed**, the integral gives **distance traveled**.

Geometric meaning

The integral is the **area under a curve**.

Set is a data structure where no duplicate elements are present. Generally, in arrays, we can store duplicate elements, but we can't store duplicate elements in set

https://www.reddit.com/r/learnpython/comments/eb57p0/what_is_the_point_of_name_main_in_python_programs/

`__name__` is a special Python variable name. As I understand, it is used to check if you're currently running through a line in the top-level (`__main__`) script, or if you're running through lines in the lower-level (imported) script.

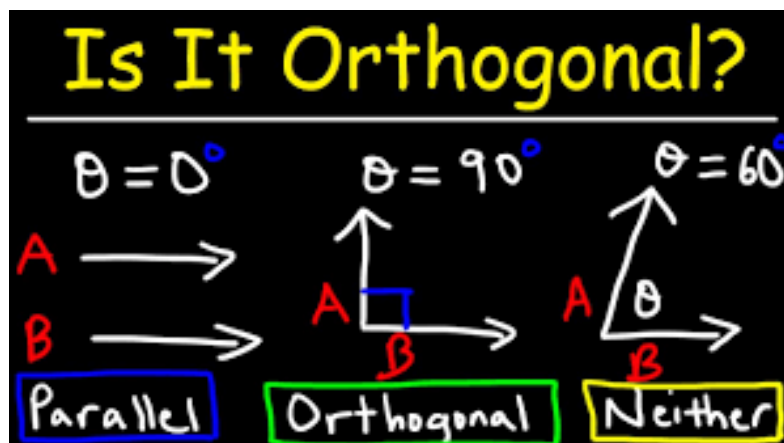
Benefits:

- Clean program entry point
- Prevents unwanted execution during imports
- Improves modular programming

In programming, **aliasing** occurs when a single memory location is accessible through more than one name or reference in a program

Orthonormal and orthogonal

A set of vectors is said to be **orthogonal** if every pair of vectors in the set is orthogonal (the dot product is 0). The set is **orthonormal** if it is orthogonal and each vector is a unit vector (norm equals 1).



Principal Component Analysis

PCA is defined as an **orthogonal linear transformation** on a real **inner product space** that transforms the data to a new **coordinate system** such that the greatest variance by some scalar projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on. ^[13]

ReLU

In the context of artificial neural networks, the **rectifier** or **ReLU (rectified linear unit) activation function**^{[1][2]} is an activation function defined as the non-negative part of its argument, i.e., the ramp function:

$$\text{ReLU}(x) = x^+ = \max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0, \\ 0 & x \leq 0 \end{cases}$$

where x is the input to a neuron. This is analogous to [half-wave rectification](#) in [electrical engineering](#).

ReLU	GELU
x = -2: 0	-0.045 ← small but non-zero
x = -1: 0	-0.169 ← preserves some info
x = -0.5: 0	-0.154 ← smooth transition
x = 0: 0	0
x = 0.5: 0.5	0.345
x = 1: 1	0.841
x = 2: 2	1.954

The landscape — all major options

Framework	By	Best for	Style
LangChain	LangChain Inc	General purpose, RAG, agents	Very modular, lots of abstractions
LangGraph	LangChain Inc	Complex agent workflows, cycles	Graph based, explicit state
LlamaIndex	LlamaIndex Inc	RAG, document processing	Document focused
AutoGen	Microsoft	Multi-agent systems	Conversation based
CrewAI	CrewAI Inc	Multi-agent with roles	Role based agents
Haystack	deepset	Production NLP pipelines	Pipeline focused

DSPy	Stanford	Optimizing LLM prompts automatically	Research oriented
Google ADK	Google	Google ecosystem agents	Similar to LangChain
Raw API	—	Full control, simple tasks	No abstractions

Retrieval augmented generation, or RAG, is an architecture for optimizing the performance of an artificial intelligence (AI) model by connecting it with external knowledge bases.